Module 13 – Python Fundamentals (Theory Answers)

- **Introduction to Python**

1. **Introduction to Python and its Features (simple, high-level, interpreted language).**

Python is a simple and easy-to-learn programming language. It is high-level, so we don't need to worry about complex system details. It is interpreted, meaning code runs line by line, which makes debugging easier.

2. **History and evolution of Python.**

Python was created by Guido van Rossum in 1991. It started as a simple scripting language and later became very powerful. Today it is widely used in web development, AI, and data science.

3. **Advantages of using Python over other programming languages.**

Python is easy to read and write, which saves time. It has a large library support and works on multiple platforms. It is widely used in modern technologies like AI and automation.

4. **Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).**

We can install Python from its official website. Then we can use tools like VS Code or PyCharm to write and run code. These tools make coding easier with features like auto-suggestions.

5. **Writing and executing your first Python program.**

The first program is usually "Hello World". We write the code using print() function and run it in terminal or IDE. It helps us understand basic syntax.

- ***Programming Style***

1. **Understanding Python's PEP 8 guidelines.**

PEP 8 is a set of rules for writing clean Python code. It helps make code readable and standard. Following it makes teamwork easier.

2. **Indentation, comments, and naming conventions in Python.**

Python uses indentation to define blocks of code. Comments are used to explain code for better understanding. Naming should be simple and meaningful.

3. **Writing readable and maintainable code.**

Code should be simple and properly formatted. Use clear variable names and proper spacing. This helps others understand and update the code easily.

- ***Core Python Concepts***

1. **Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets.**

Python supports different data types like int, float, and string. It also has collections like list, tuple, dictionary, and set. These help store and manage data efficiently.

2. **Python variables and memory allocation.**

Variables store data in memory. Python automatically manages memory, so we don't need to define type manually. It makes coding faster.

3. **Python operators: arithmetic, comparison, logical, bitwise.**

Operators are used to perform operations on data. Arithmetic is for calculations, comparison checks values, logical combines conditions, and bitwise works at binary level.

- ***Conditional Statements***

1. **Introduction to conditional statements: if, else, elif.**

Conditional statements help make decisions in code. If checks condition, else runs when condition is false, and elif checks multiple conditions.

2. **Nested if-else conditions.**

Nested if means using one if inside another. It helps check multiple conditions step by step. Useful for complex decision making.

- ***Looping (For, While)***

1. **Introduction to for and while loops.**

Loops are used to repeat tasks. For loop runs for a fixed number of times, while loop runs until condition becomes false.

2. **How loops work in Python.**

Loops execute code repeatedly based on condition. Python uses indentation to define loop body. It makes repetition easy.

3. **Using loops with collections (lists, tuples, etc.).**

We can use loops to access elements of lists or tuples. It helps process multiple values easily. Very useful in real programs.

- ***Generators and Iterators***

1. **Understanding how generators work in Python.**

Generators are functions that return values one by one using yield. They save memory by not storing all values at once.

2. **Difference between yield and return.**

Return ends the function and gives value once. Yield pauses the function and continues later. It is used in generators.

3. **Understanding iterators and creating custom iterators**.

Iterators are objects used to loop over data. We can create custom iterators using iter() and next() methods.

- ***Functions and Methods***

1. **Defining and calling functions in Python.**

Functions are blocks of code that perform a task. We define using def and call by function name. It helps reuse code.

2. **Function arguments (positional, keyword, default).**

Arguments pass values to functions. Positional follows order, keyword uses names, and default gives fixed value if not passed.

3. **Scope of variables in Python**.

Scope defines where variable can be used. Local variables work inside function, global variables work everywhere.

4. **Built-in methods for strings, lists, etc.**

Python provides many built-in methods like upper(), append(), etc. They help perform operations easily without writing extra code.

- ***Control Statements (Break, Continue, Pass)***

1. **Understanding the role of break, continue, and pass in Python loops**.

Break stops the loop completely. Continue skips current iteration. Pass does nothing but is used as placeholder.

- ***String Manipulation***

1. **Understanding how to access and manipulate strings.**

Strings can be accessed using index. We can change or modify them using different methods.

2. **Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).**

We can join strings using + and repeat using *. Methods like upper() and lower() change text format.

3. **String slicing**.

Slicing is used to get part of a string. It uses start and end index. Very useful for extracting data.

- ***Advanced Python (map(), reduce(), filter(), Closures and Decorators)***

1. **How functional programming works in Python**.

Functional programming focuses on functions and data processing. It avoids changing data directly.

2. **Using map(), reduce(), and filter() functions for processing data.**

Map applies function to all elements. Filter selects elements based on condition. Reduce combines elements into single value.

3. **Introduction to closures and decorators.**

Closures are functions inside functions that remember values. Decorators are used to modify functions without changing their code.